Northeastern University

Class Assignment 3: EECE 7205

Professor: Dr. Fatema Nafa

Rohini Bidnurmath
NUID: 003916201

Catalina Buitano
NUID: 002330106

October 2026

Contents

Setup and Conventions 2

1 Case 1: Deleting the Root 2

2 Case 2: Deleting a Leaf 5

3 Case 3: Deleting a Node with One Child 8

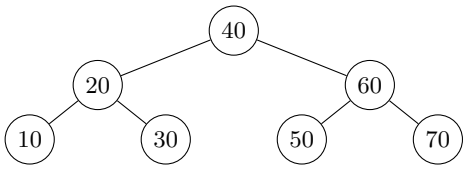
4 Case 4: Deleting a Node with Two Children 11

5 Case 5: The Key Is Not in the Tree 13

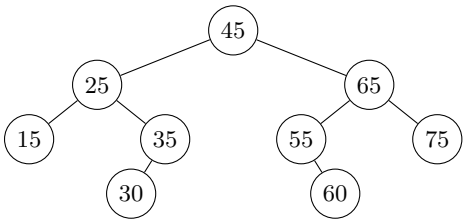
6 Bonus: One Function for Every Case 16

Setup and Conventions

Tree T is built by inserting $40, 20, 60, 10, 30, 50, 70$. Tree P is built by inserting $45, 25, 65, 15, 35, 55, 75, 30, 60$. The height h of a tree is the number of edges on its longest root-to-leaf path. In every figure, red bold values mark the node or path that the question is about.



Tree T
Inorder: 10 20 30 40 50 60 70



Tree P
Inorder: 15 25 30 35 45 55 60 65 75

Figure 1: The two starting trees.

1 Case 1: Deleting the Root

Rules used: B5 (two children) and B6 (root).

Question 1

The search for the inorder successor of the root 40 goes right once and then left as far as possible.

Step	Node visited	Move	Reason
1	60	go right from 40	start of the right subtree
2	50	go left from 60	60 → left = 50
3	(stop)	—	50 → left = NULL

The nodes visited are **60 and 50**. The successor is **50** and its parent is **60**.

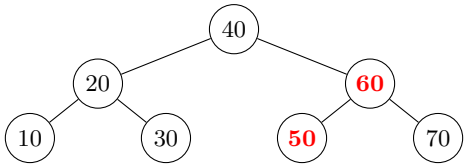


Figure 2: Tree T with the successor search path highlighted.

Question 2

After copying 50 into the root and unlinking the old leaf 50 (60 → left = NULL):

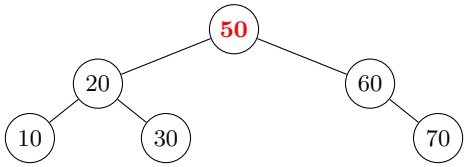


Figure 3: Tree T after deleting 40.

- Inorder: `10 20 30 50 60 70`

- Preorder: `50 20 10 30 60 70`

Why B7 holds. The inorder traversal is strictly increasing. It is exactly the old sequence `10 20 30 40 50 60 70` with 40 removed. For a binary tree, an increasing inorder traversal is equivalent to the BST property holding at every node, so the tree is still a valid BST. Concretely, 50 is larger than everything on the left (10, 20, 30) and smaller than everything on the right (60, 70).

Question 3

Changed lines (predecessor version, saved as `case1_q3.cpp`):

```
Node* parent = root;
Node* pred = root->left;          // go left once (20)
while (pred->right != nullptr)    // then right as far as possible
{
    parent = pred;
    pred = pred->right;           // pred = 30, parent = 20
}
root->data = pred->data;          // root now holds 30
if (parent == root)
    parent->left = pred->left;
else
    parent->right = pred->left;   // 20->right = NULL
delete pred;
```

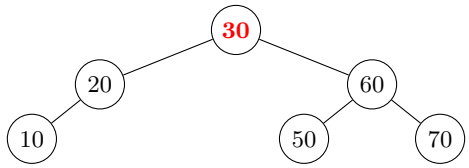


Figure 4: Tree T after deleting 40 with the inorder predecessor.

The **new root is 30**. The inorder output is **unchanged**: `10 20 30 50 60 70`. The predecessor (30) and the successor (50) are both neighbours of 40 in sorted order. Whichever one we move up, the remaining values are the same set, and the inorder traversal of any valid BST lists its keys in sorted order. Only the *shape* of the tree changes (preorder becomes `30 20 10 60 50 70`).

Question 4

Tree P, delete the root 45. Going right once gives 65, then left gives 55; 55 $\rightarrow$ left = NULL, so we stop. The **successor is 55** and its parent is 65.

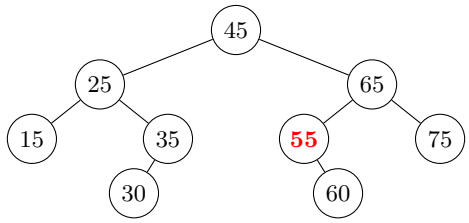


Figure 5: Tree P with the successor 55 highlighted.

The successor node is removed by **B4 (one child)**, not B3, because 55 is *not* a leaf: it has a right child, 60. That child takes its place (`parent->left = succ->right`, so 65 $\rightarrow$ left = 60).

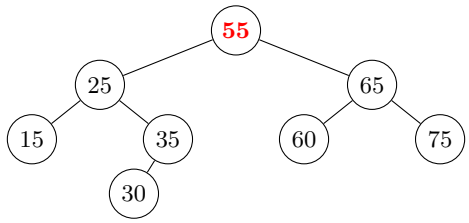


Figure 6: Tree P after deleting 45.

Inorder: `15 25 30 35 55 60 65 75` (sorted, so B7 holds).

Program output (Case 1)

Expected output of `case1.cpp`:

```
Before: 10 20 30 40 50 60 70 (root = 40)
After:  10 20 30 50 60 70 (root = 50)
```

Expected output of `case1_q3.cpp`: `After:  10 20 30 50 60 70 (root = 30)`.

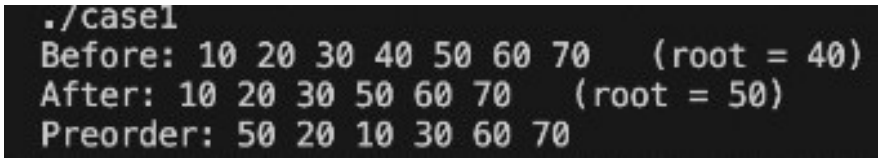


Figure 7: Screenshot for Case 1 Original Code

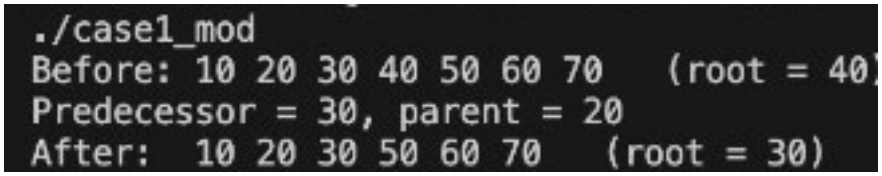


Figure 8: Screenshot for Case 1 Modified Code

2 Case 2: Deleting a Leaf

Rules used: B1 (search) and B3 (leaf).

Question 1

Trace of the search loop for $key = 10$ in tree T:

Node visited	Comparison	Direction	parent after the step
40	$10 < 40$	left	40
20	$10 < 20$	left	20
10	$10 = 10$	found, stop	20

The parent is **20**, and the deleted node 10 is its **left** child, so `20->left = nullptr`.

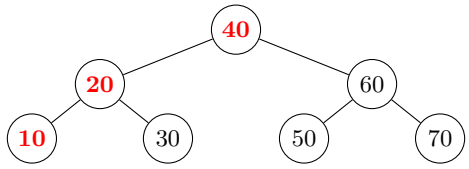


Figure 9: Search path for 10 in T.

Question 2

Changed line (saved as `case2_q2.cpp`):

```
1 int key = 70;      // was 10
```

The path is $40 \to 60 \to 70$. Both comparisons go right ($70 > 40$, $70 > 60$), so 70 is the **right** child of 60 and the pointer set to NULL is `60->right`. The program prints:

```
After:  10 20 30 40 50 60 (root = 40)
```

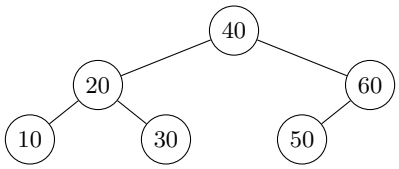


Figure 10: Tree T after deleting 70.

Question 3

The leaves of P are **15, 30, 60 and 75**. (35 has the left child 30 and 55 has the right child 60, so they are not leaves.)

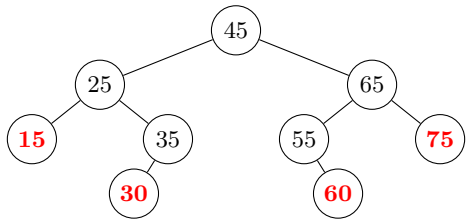


Figure 11: Tree P with its leaves highlighted.

Deleting 30: the path is $45 \to 25 \to 35 \to 30$, the parent is 35 and 30 is its left child, so $35 \to \text{left} = \text{NULL}$.

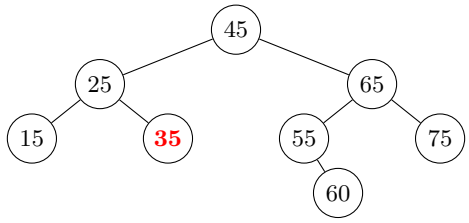


Figure 12: Tree P after deleting 30. Node 35 has just become a leaf.

The node that has just become a leaf is **35**.

Question 4

The search examines one node per level, so in the worst case it visits the $h+1$ nodes on a root-to-leaf path, i.e. at most $h+1$ **key comparisons**, which is $O(h)$. Unlinking the leaf is $O(1)$.

- Tree T: $h = 2$ (at most 3 comparisons).
- Inserting $10, 20, 30, 40, 50, 60, 70$ in that order gives a chain, so $h = 6$ (at most 7 comparisons).

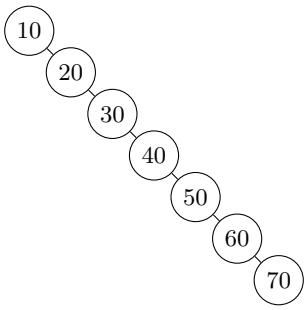


Figure 13: Tree built from sorted input ($h = 6$).

Deleting a leaf takes time proportional to the height, not to the number of nodes: $O(\log n)$ for a balanced tree but $O(n)$ for a degenerate (chain) tree.

Program output (Case 2)

```
Before: 10 20 30 40 50 60 70 (root = 40)
After:  20 30 40 50 60 70 (root = 40)
```

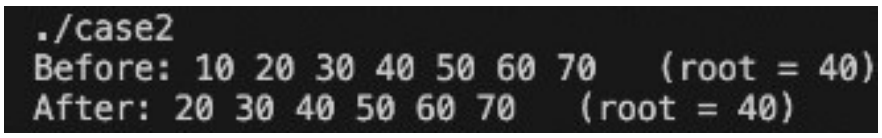


Figure 14: Screenshot for Case 2 Original Code

```
./case2_mod
Before: 10 20 30 40 50 60 70   (root = 40)
After:  10 20 30 40 50 60   (root = 40)
```

Figure 15: Screenshot for Case 2 Modified Code

3 Case 3: Deleting a Node with One Child

Rules used: B1, B4 (and B6 for the root).

Question 1

In tree P exactly two nodes have one child:

- **35** has only a *left* child (30).
- **55** has only a *right* child (60).

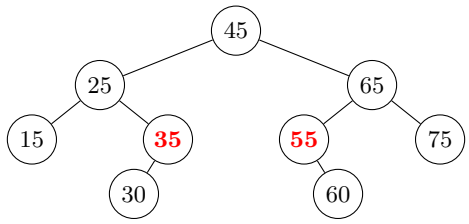


Figure 16: Tree P with the one-child nodes highlighted.

Question 2

Delete 35 from P (path $45 \to 25 \to 35$). Parent: **25**. The child that moves up: **30**. Since $35 > 25$, 35 is the right child of 25, so the pointer that changes is `25->right = 30`.

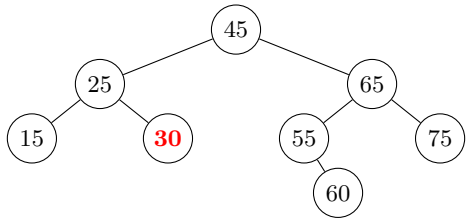


Figure 17: Tree P after deleting 35.

Inorder: `15 25 30 45 55 60 65 75`.

Question 3

Delete 55 from the original P (path $45 \to 65 \to 55$). Parent: **65**. The child that moves up: **60**. The pointer that changes is `65->left = 60`.

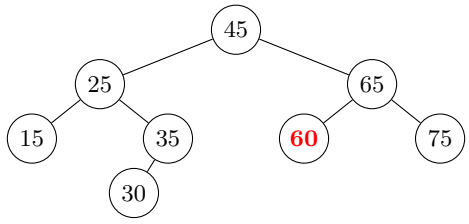


Figure 18: Tree P after deleting 55.

Inorder: `15 25 30 35 45 60 65 75`.

The program uses `parent->left` this time because $55 < 65$: 55 is the *left* child of 65. In Question 2, $35 > 25$ made 35 a right child, so `parent->right` was used. The test `if (parent->left == cur)` chooses the correct side automatically.

Question 4

Insert $10, 20, 30$ (a right-leaning chain) and delete the root 10.



Figure 19: Deleting the root 10 from the chain $10 \to 20 \to 30$.

Rule B6 says that for a root with 0 or 1 child the root pointer itself changes. The **new root is 20**.

What goes wrong in the unchanged `case3.cpp`. The root has no parent, so the search loop never executes `parent = cur` and `parent` stays `nullptr`. The line `if (parent->left == cur)` then dereferences a null pointer, which is undefined behaviour (a segmentation fault). Even without the crash, `root` would still point to the freed node, because nothing updates it.

Fix (saved as `case3_q4.cpp`):

```
Node* child = (cur->left != nullptr) ? cur->left : cur->right;
if (parent == nullptr)          // B6: deleting the root
    root = child;               // the root pointer itself changes
else if (parent->left == cur)
    parent->left = child;
else
    parent->right = child;
delete cur;
```

Output: `Before: 10 20 30 (root = 10)`, `After: 20 30 (root = 20)`.

Changes used for Questions 2 and 3 (tree P)

```
// build P instead of T
Node* root = nullptr;
int keys[] = {45, 25, 65, 15, 35, 55, 75, 30, 60};
for (int k : keys) root = insertNode(root, k);
// ...
int key = 35;       // Question 2;  use 55 for Question 3
```

Program output (Case 3)

```
Before: 15 25 30 35 45 55 60 65 75 (root = 45)
After (delete 35): 15 25 30 45 55 60 65 75 (root = 45)
After (delete 55): 15 25 30 35 45 60 65 75 (root = 45)
```

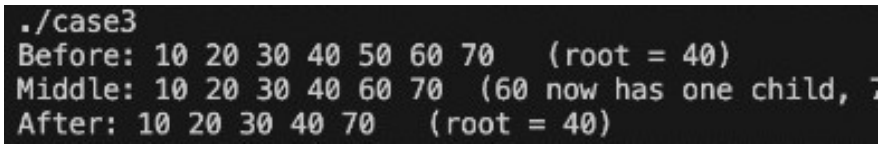


Figure 20: Screenshot for Case 3 Original Code

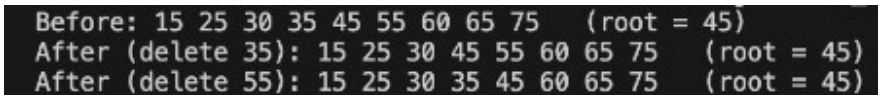


Figure 21: Screenshot for Case 3 Q2 Q3 Modified Code



Figure 22: Screenshot for Case 3 Q4 Modified Code

4 Case 4: Deleting a Node with Two Children

Rules used: B1 and B5 (then B3 or B4 for the successor).

Question 1

Changed line: `int key = 60;` (was 20). The search finds 60 as the root's right child. Then `succ = cur->right` = 70. Since 70 → left = NULL, the `while` loop does not run, so `parent` remains `cur`.

- After the loop: **succ = 70, parent = 60** (the node being deleted).
- The branch that runs is `if (parent == cur) parent->right = succ->right;`. We never walked left, so the successor is the *right* child of the node, and the link to repair is `parent->right` (here NULL).
- Printed: `After:  10 20 30 40 50 70 (root = 40).`

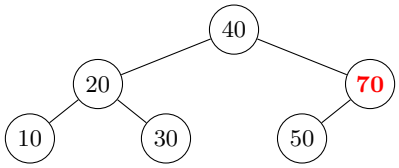


Figure 23: Tree T after deleting 60.

Question 2

The successor is found by going right once and then left until there is no left child, so it is the smallest key of the right subtree. If it had a left child, that child would hold a smaller key that is still larger than the deleted node, contradicting "smallest". In practice the loop would simply not have stopped there. So the successor has at most a right child.

If it *could* have a left child, step 4 (`parent->left = succ->right`) would only reattach the right subtree and the left subtree of `succ` would be cut off and lost (a memory leak, and missing keys). Removal would no longer be a simple B3/B4 case.

Question 3

Delete 25 from P. It has two children (15 and 35). Go right to 35, then left to 30; 30 → left = NULL.

- Successor: **30**. Its parent: **35**.
- Since parent ≠ cur, the `else` branch runs: `35->left = succ->right = NULL`.
- The node that held 25 now holds 30.

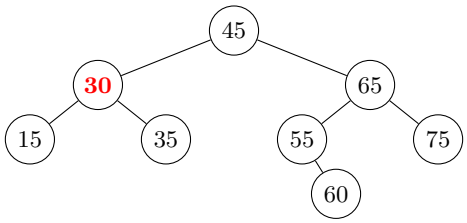


Figure 24: Tree P after deleting 25 (successor).

Inorder: `15 30 35 45 55 60 65 75.`

Question 4

Using the inorder predecessor: go left once to 15; 15 → right = NULL, so the predecessor is 15 and its parent is the node being deleted. Since parent = cur, the link set is `parent->left = pred->left` = NULL. Code

used:

```
Node* parent = cur;
Node* pred = cur->left;             // 15
while (pred->right != nullptr)
{
    parent = pred;
    pred = pred->right;
}
cur->data = pred->data;             // the node that held 25 now holds 15
if (parent == cur)
    parent->left = pred->left;      // taken here: 25's left becomes NULL
else
    parent->right = pred->left;
delete pred;
```

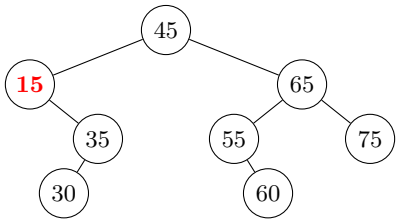


Figure 25: Tree P after deleting 25 (predecessor).

Inorder: `15 30 35 45 55 60 65 75`.

It is **not the same tree** as in Question 3 (there the node is 30 with children 15 and 35; here it is 15 with only a right subtree). However, **both are valid BSTs**, and they have the same inorder traversal. Each node is larger than everything in its left subtree and smaller than everything in its right subtree.

Program output (Case 4)

```
Before: 10 20 30 40 50 60 70 (root = 40)
After (key = 20): 10 30 40 50 60 70 (root = 40)
After (key = 60): 10 20 30 40 50 70 (root = 40)
```

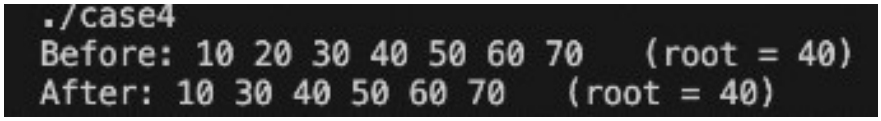


Figure 26: Screenshot for Case 4 Original Code

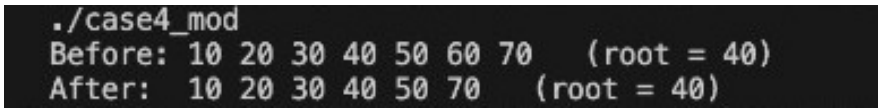


Figure 27: Screenshot for Case 4 modified Code

5 Case 5: The Key Is Not in the Tree

Rules used: B1 and B2.

Question 1

Search for 25 in T:

Node	Comparison	Direction
40	$25 < 40$	left
20	$25 > 20$	right
30	$25 < 30$	left
NULL	—	stop

The search makes **3 comparisons** and stops at the **left link of node 30**.

Question 2

Key	Path	Stops at	Comparisons
5	40 (L), 20 (L), 10 (L)	left link of 10	3
45	40 (R), 60 (L), 50 (L)	left link of 50	3
75	40 (R), 60 (R), 70 (R)	right link of 70	3

No key needs more than the others: **all three need 3 comparisons** (as does 25). Tree T is perfectly balanced with $h = 2$, so every NULL link lies at depth 3, and an unsuccessful search always makes $h + 1 = 3$ comparisons. In general the worst case is $h + 1$ comparisons.

Question 3

Changed code (saved as `case5_q3_q4.cpp`). The parent is remembered during the search, and the key is inserted at the NULL link where the search stopped:

```
Node* parent = nullptr;                 // remember the parent
Node* cur = root;
int key = 25;
while (cur != nullptr && cur->data != key)
{
    parent = cur;
    if (key < cur->data) cur = cur->left;
    else cur = cur->right;
}
if (cur == nullptr)
{
    Node* n = new Node(key);            // insert at the NULL link
    if (key < parent->data) parent->left = n;   // 30->left = 25
    else parent->right = n;
}
```

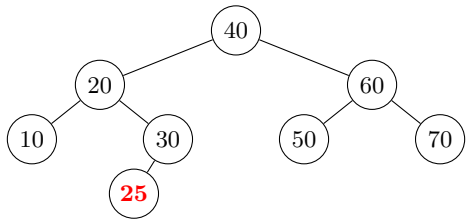


Figure 28: Tree T after inserting 25.

Inorder: `10 20 25 30 40 50 60 70`.

Question 4

If `root = nullptr`, the statement that prints `root->data` dereferences a null pointer, which is undefined behaviour (a segmentation fault). The fix is to check first:

```
if (root == nullptr)
{
    cout << "the tree is empty" << endl;
    return 0;
}
```

This case is covered by **rule B2**: the search reaches NULL immediately, so the key is not in the tree and nothing changes.

Program output (Case 5)

```
Before: 10 20 30 40 50 60 70 (root = 40)
25 is not in the tree: nothing to delete
After:  10 20 30 40 50 60 70 (root = 40)
```

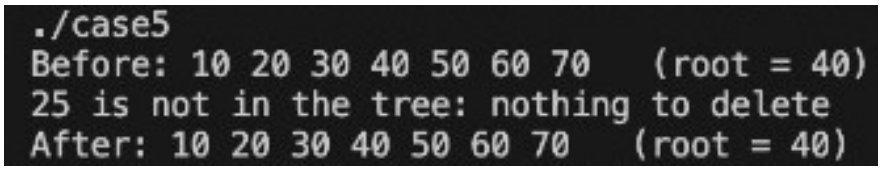


Figure 29: Screenshot for Case 5 Original Code

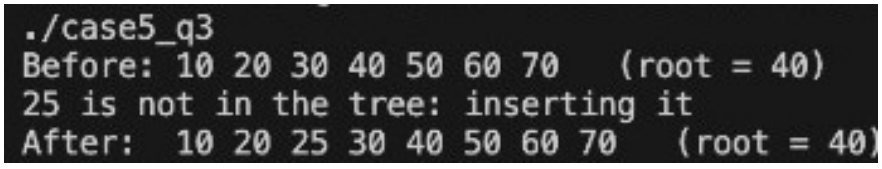


Figure 30: Screenshot for Case 5 Modified Q3 Code

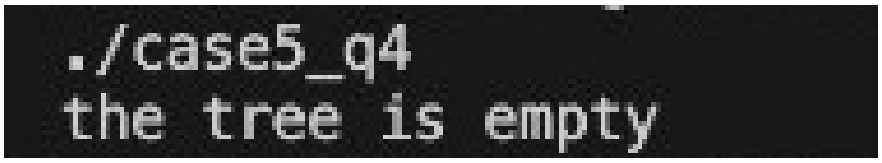


Figure 31: Screenshot for Case 5 Modified Q4 Code

6 Bonus: One Function for Every Case

Algorithm 1 Delete a key from a BST

Input: root of a BST, key to delete
Output: root of the tree after deletion
Initial call: DeleteNode($root, key$)

```
1: parent ← NULL; cur ← root
2: while cur ≠ NULL and cur.data ≠ key do                          ▷ B1
3:     parent ← cur
4:     if key < cur.data then
5:         cur ← cur.left
6:     else
7:         cur ← cur.right
8:     end if
9: end while
10: if cur = NULL then
11:     return root
12: end if                                                         ▷ B2: not found
13: if cur.left ≠ NULL and cur.right ≠ NULL then                   ▷ B5: two children
14:     sp ← cur; succ ← cur.right
15:     while succ.left ≠ NULL do
16:         sp ← succ; succ ← succ.left
17:     end while
18:     cur.data ← succ.data
19:     if sp = cur then
20:         sp.right ← succ.right
21:     else
22:         sp.left ← succ.right
23:     end if
24:     free succ; return root
25: end if
26: child ← cur.left if cur.left ≠ NULL else cur.right              ▷ B3/B4
27: if parent = NULL then
28:     root ← child                                               ▷ B6
29: else if parent.left = cur then
30:     parent.left ← child
31: else
32:     parent.right ← child
33: end if
34: free cur; return root
```

C++ implementation:

```
1  Node* deleteNode(Node* root, int key)
2  {
3      Node* parent = nullptr;
4      Node* cur = root;
5      while (cur != nullptr && cur->data != key)      // B1
6      {
7          parent = cur;
8          if (key < cur->data) cur = cur->left;
9          else cur = cur->right;
10     }
11     if (cur == nullptr) return root;                // B2
```

```
12
13      if (cur->left != nullptr && cur->right != nullptr)   // B5
14      {
15          Node* sp = cur;
16          Node* succ = cur->right;
17          while (succ->left != nullptr) { sp = succ; succ = succ->left; }
18          cur->data = succ->data;
19          if (sp == cur) sp->right = succ->right;
20          else sp->left = succ->right;
21          delete succ;
22          return root;
23      }
24
25      Node* child = (cur->left != nullptr) ? cur->left : cur->right;  // B3, B4
26      if (parent == nullptr) root = child;            // B6
27      else if (parent->left == cur) parent->left = child;
28      else parent->right = child;
29      delete cur;
30      return root;
31  }
```

Test on tree T

Deleting $40, 10, 60, 25$ one after another:

Delete	Case	Inorder afterwards	Root
40	two children (successor 50)	10 20 30 50 60 70	50
10	leaf	20 30 50 60 70	50
60	one child (70)	20 30 50 70	50
25	not found, no change	20 30 50 70	50

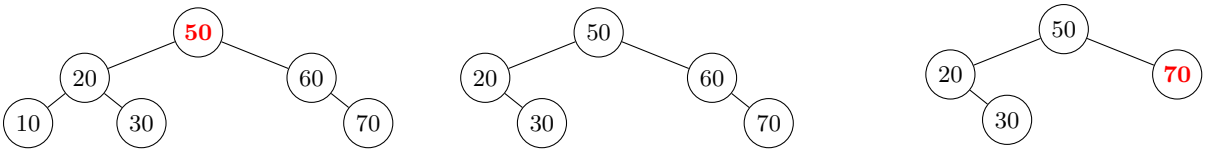


Figure 32: Tree T after each deletion.

```
Start: 10 20 30 40 50 60 70
Delete 40: 10 20 30 50 60 70
Delete 10: 20 30 50 60 70
Delete 60: 20 30 50 70
Delete 25: 20 30 50 70
```

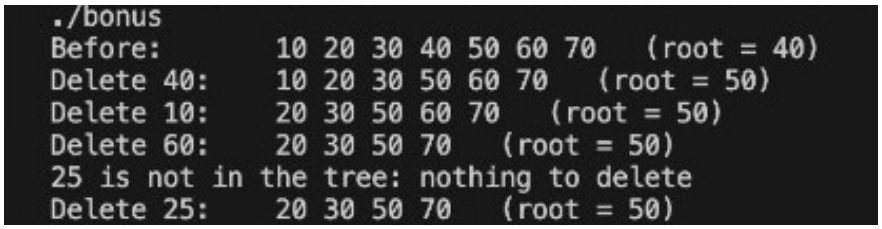


Figure 33: Screenshot for Bonus Code

Time complexity. The search walks down one path, which is at most $h + 1$ nodes. For two children, the successor search continues down a single path below the node, so the total path length is still at most $h + 1$. All pointer updates are $O(1)$. Hence deletion is $O(h)$: $O(\log n)$ for a balanced tree and $O(n)$ in the worst case (a chain). The space used is $O(1)$.